

Đặc tả: Xác thực và Phân quyền (Authentication & Authorization)

1. Mô tả

Tính năng quản lý danh tính người dùng của UniHub được chia làm 2 tầng rõ rệt:

- **Xác thực (Authentication - AuthN):** Tích hợp OAuth 2.0 (Google Sign-In) để xác minh danh tính người dùng thông qua email trường đại học (VD: @student.truong.edu.vn), loại bỏ hoàn toàn việc lưu trữ mật khẩu nhạy cảm trên hệ thống.
- **Phân quyền (Authorization - AuthZ):** Áp dụng mô hình RBAC (Role-Based Access Control) thông qua JWT. Hệ thống cấp phát Access Token (thời hạn ngắn) và Refresh Token (thời hạn dài) để phân luồng người dùng vào 3 nhóm quyền cốt lõi: STUDENT, ORGANIZER, và CHECKIN_STAFF.

2. Luồng chính (Happy Path)

Luồng 2.1: Xác thực Đăng nhập (Google OAuth & JWT Generation)

Bối cảnh: Sinh viên (hoặc Ban tổ chức) lần đầu truy cập vào hệ thống UniHub. Đối với nhân sự Check-in sử dụng Mobile App, thao tác Đăng nhập và lấy JWT Token bắt buộc phải thực hiện vào thời điểm đầu ngày (khi có kết nối WiFi) trước khi đi vào khu vực mất mạng để quét QR.

B1. Trigger: Người dùng bấm nút “Đăng nhập bằng Google” trên giao diện Web/Mobile.

B2. Xác thực bên thứ 3: Cửa sổ Google OAuth hiện ra, người dùng chọn tài khoản email trường và đồng ý cấp quyền. Google trả về cho Frontend một chuỗi Google Credential Token.

B3. Gửi Token lên Server: Frontend gửi chuỗi Token này lên API POST /api/auth/google.

B4. Xác minh & Đối chiếu: Backend sử dụng thư viện google-auth-library để xác minh Token. Nếu hợp lệ, trích xuất email và truy vấn vào bảng Users trong PostgreSQL:

- (Lưu ý: Tài khoản STUDENT đã được Worker tạo sẵn qua luồng CSV Sync mỗi đêm, tài khoản ORGANIZER đã được seed sẵn).
- Nếu email TỒN TẠI trong DB -> Trích xuất user_id và role.

B5. Cấp phát JWT: Backend tạo 2 loại token:

- **Access Token:** Chứa payload { user_id, role, exp }, ký bằng JWT_SECRET, thời hạn 15 phút.
- **Refresh Token:** Chuỗi ngẫu nhiên lưu vào DB, thời hạn 7 ngày.

B6. Phản hồi: Backend trả Access Token trực tiếp qua body JSON (để Frontend lưu vào biến memory), và tự động cài Refresh Token vào trình duyệt của người dùng thông qua **HttpOnly Cookie**.

Luồng 2.2: Phân quyền gọi API (RBAC Middleware)

Bối cảnh: Sinh viên gọi API đăng ký Workshop, hoặc Admin gọi API tạo Workshop.

B1. Đính kèm Token: Frontend tự động chèn Access Token vào Header: Authorization: Bearer <Access_Token>.

B2. Chặn cửa bằng Middleware: Request đi qua một hàm Middleware bảo mật trên Node.js. Middleware này giải mã Token.

B3. Kiểm tra tính hợp lệ: Đảm bảo chữ ký (Signature) đúng và Token chưa hết hạn.

B4. Kiểm tra Quyền (Role): Middleware lấy thuộc tính role từ payload.

Ví dụ: Endpoint POST /api/workshops yêu cầu quyền ORGANIZER. Nếu role trong token là STUDENT, Middleware từ chối ngay lập tức.

B5. Pass: Nếu hợp lệ, Middleware gán req.user = decoded_payload và cho phép request đi tiếp vào Controller xử lý logic.

3. Kịch bản lỗi (Edge Cases & Exception Handling)

3.1. Email hợp lệ nhưng chưa có trong hệ thống

- **Trigger:** Tân sinh viên dùng email trường đăng nhập bằng Google, nhưng đêm qua tiến trình CSV Sync chưa chạy tới dữ liệu của sinh viên này.

- **Xử lý:** Ở bước đối chiếu (B4), Backend không tìm thấy email trong bảng Users. Từ chối cấp JWT, trả về HTTP 403 Forbidden kèm thông báo: “Tài khoản của bạn chưa được đồng bộ vào hệ thống. Vui lòng thử lại vào sáng mai.”

3.2. Hết hạn Access Token (Silent Refresh)

- **Trigger:** Sau 15 phút sử dụng, Access Token của sinh viên hết hạn. Sinh viên bấm xem danh sách vé, API trả về 401 Unauthorized.
- **Xử lý:** Frontend (Axios Interceptor) “bắt” được lỗi 401. Nó sẽ ngầm gọi API POST /api/auth/refresh-token. Backend sẽ tự động đọc **HttpOnly Cookie** chứa Refresh Token, kiểm tra tính hợp lệ và cấp lại Access Token mới. Frontend tự động gắn token mới này và gửi lại request đang bị kẹt. Người dùng hoàn toàn không cảm nhận được sự gián đoạn.

3.3. Cố tình giả mạo quyền hạn (JWT Tampering)

- **Trigger:** Một sinh viên hiểu biết về IT lên trang jwt.io, tự decode Access Token của mình, sửa chữ STUDENT thành ORGANIZER, mã hóa lại và gửi lên Backend để hack hệ thống.
- **Xử lý:** Ở bước kiểm tra chữ ký (Signature Validation), Backend phát hiện Token đã bị thay đổi payload nhưng không có JWT_SECRET hợp lệ để ký lại. Trả về ngay 401 Unauthorized và ghi log an ninh giám sát IP của người dùng này.

3.4. Refresh Token bị đánh cắp (Token Hijacking)

- **Trigger:** Một mã độc XSS chạy trên trình duyệt cố gắng đánh cắp Refresh Token của Admin.
- **Xử lý:** Do Refresh Token được bảo vệ bởi cờ HttpOnly, mã JavaScript/XSS của hacker hoàn toàn **không thể** đọc được cookie này. Lỗ hổng bị vô hiệu hóa từ trong trứng nước.

4. Ràng buộc (Constraints)

- **Bảo mật Cookie:** Refresh Token Cookie bắt buộc phải được set các cờ: HttpOnly = true (Chống XSS), Secure = true (Chỉ chạy trên HTTPS), SameSite = Strict (Chống CSRF). (Lưu ý: Do hệ thống triển khai ở môi trường Local (Docker) để chấm đồ án, cờ Secure = true có thể được tạm thời cấu hình thành false hoặc sử dụng chứng chỉ SSL tự ký (self-signed) cho localhost để trình duyệt cho phép lưu Cookie.)

- **Nguyên tắc “Default Deny”:** Mọi API endpoint (ngoại trừ /api/auth/* và GET /api/workshops public) đều mặc định bị khóa. Dev phải chủ động khai báo Middleware requireRole([...]) thì API mới được mở cho Role đó.
- **Quản lý phiên (Session Revocation):** Khi người dùng bấm “Đăng xuất” hoặc Admin khóa tài khoản (Deactivate), Backend phải lập tức xóa Refresh Token trong Database và ra lệnh xóa Cookie trên trình duyệt.

5. Tiêu chí chấp nhận (Acceptance Criteria)

- **Test Case 1 (Login Flow):** Gọi API auth với một Google Token hợp lệ của email đã có trong DB. Server trả về mã 200, trong Header Set-Cookie có chứa Refresh Token với cờ HttpOnly.
- **Test Case 2 (RBAC Security):** Dùng Access Token có role STUDENT gọi API DELETE /api/workshops/1. Server trả về chính xác mã lỗi 403 Forbidden (Không có quyền truy cập).
- **Test Case 3 (Token Expiration):** Đợi 16 phút để Access Token hết hạn, dùng token này gọi API. Server trả về 401 Unauthorized (Token expired).
- **Test Case 4 (Refresh Mechanism):** Gọi API refresh-token với cookie hợp lệ. Server cấp Access Token mới. Gọi lại API bằng token mới, server trả về dữ liệu bình thường.